

Name: Bhagyasri Dommaraju

Copart Conversational Search: Architecture and Key Design Decisions

How I built the Copart Conversational Search prototype

I designed this project as a working end-to-end prototype that feels conversational to the user, while keeping the actual vehicle search predictable, transparent, and easy to test.

The AI helps understand what the user means, but deterministic application code decides which vehicles match. This separation gave me the flexibility of natural language without making the search results depend entirely on a model.

1. Conversation UI - React interface captures each request and keeps prior filters visible.
2. Intent layer - NVIDIA NIM interprets flexible language; a local parser is the fallback.
3. Filter contract - Only approved fields such as make, year, price, and fuel are accepted.
4. Search engine - TypeScript applies normalized filters to the 200-vehicle inventory.

1. Problem and approach

This project is a conversational vehicle search prototype designed to make a large inventory discoverable through natural language while preserving transparency, safety, and explainability. The challenge is not only matching text to data but turning ambiguous user intent into a structured search filter without letting untrusted input directly drive business logic.

The solution combines:

- a modern frontend for conversational discovery
- a typed filter schema that represents vehicle criteria
- an AI interpretation layer that converts prompts into structured filters
- a deterministic fallback parser for offline or failed AI cases
- a search engine that evaluates inventory against validated filters

This design is intentionally resilient: even when AI inference is unavailable, the user can continue searching using deterministic logic.

2. Architecture overview

Frontend layer:

React 19 + TypeScript + Vite provide the user experience. The user enters natural-language prompts such as "Include Honda", "Under 20k", or "No Ford and exclude flood damage". Results update with active filter chips, stateful refinement, and undo/reset controls.

Search and filter model:

The application represents search criteria in a typed object with fields such as make, model, year, price, mileage, state, fuel, transmission, drive, title, damage, color, and engine. The search logic is deterministic and inspectable, which keeps the system safe and auditable.

- **Conversational state:**
Each new message starts with the current filter object rather than starting from zero. For example, a user can ask for SUVs, then say 'only Toyota,' and then add 'under \$30,000.' I also added Undo and Reset so the conversation remains reversible.
- **Handling a possible new search:**
If a request looks like a different vehicle rather than a refinement, the application asks whether to keep the previous filters or start a new search. This avoids silently mixing unrelated requirements and makes the system's behavior easier to understand.
- Active filters are shown as chips next to the results. I wanted users and reviewers to see what the system understood instead of treating the model as a black box.
- The application carries prior filters forward, but it asks for clarification when a request may represent a different vehicle search. This balances continuity with user control.

AI refinement layer:

The server route `/api/refine` sends the current filters and the latest user query to NVIDIA NIM. The model is instructed to return JSON matching the known filter schema, and the response is normalized before applying it to the current search state.

Fallback parser:

If the NVIDIA route is unavailable or the key is missing, the app falls back to a local deterministic parser. This allows the experience to remain usable without secret configuration or remote service access.

Inventory layer:

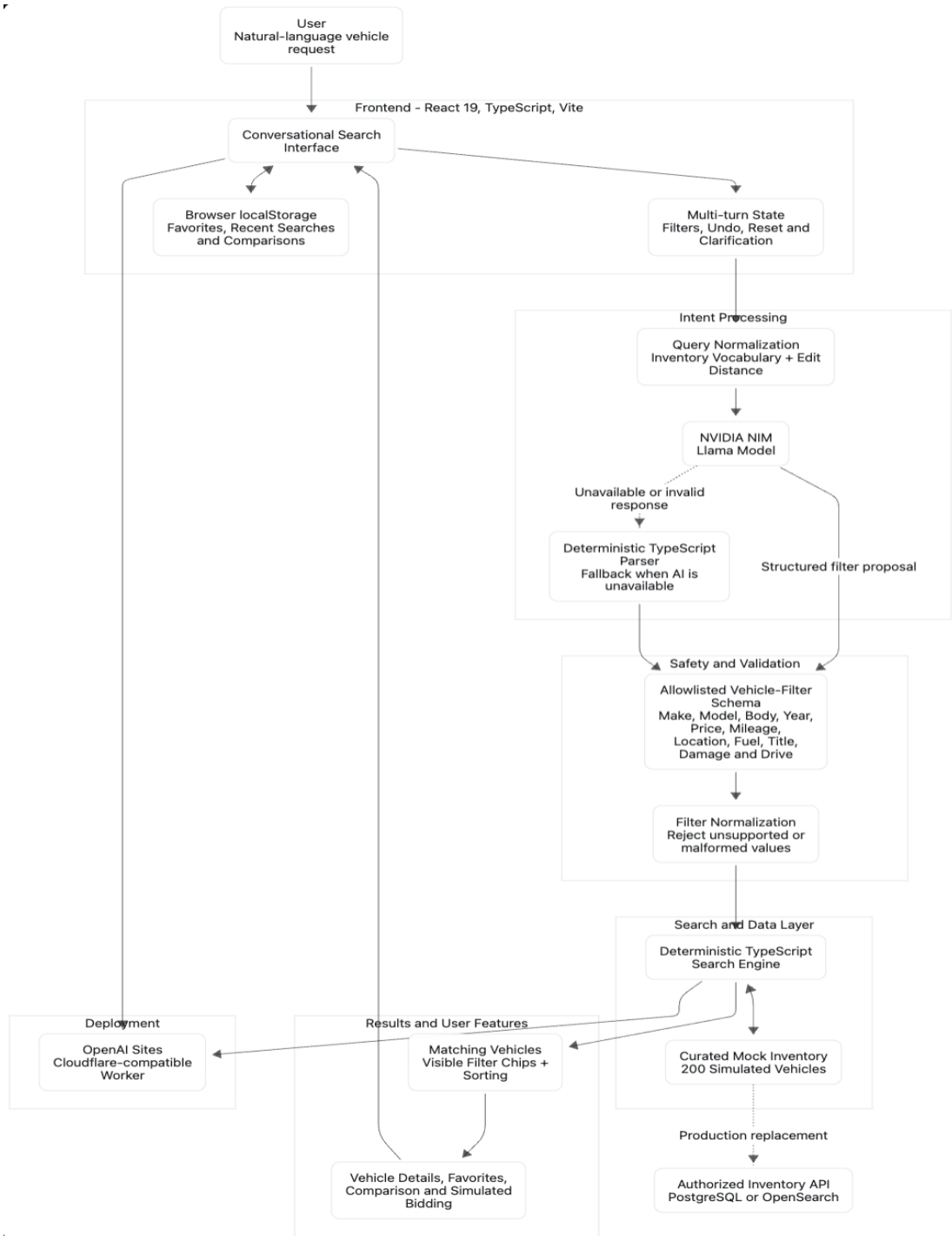
A curated in-memory inventory of simulated auction vehicles is used for demo purposes. Each vehicle contains make, model, condition, damage, price, location, title status, and representative image metadata.

Conversational AI and Inventory Architecture :

I used NVIDIA NIM with Llama 3.3 70B to interpret conversational requests into a validated vehicle-filter schema. I used a curated mock inventory rather than depending on a third-party vehicle API because the prototype required Copart-specific fields such as current bid, title status, primary damage and lot number. The data layer is isolated, so a production inventory API or OpenSearch index can replace the mock repository without changing the conversational interface

Search Query Normalization and Typo Correction:

Before processing a search, I normalize possible spelling mistakes in vehicle-related terms. I created a vocabulary from the available inventory, including makes, models, series, body styles, engines, colors, locations, fuel types, and common search words. Each user word is compared with this vocabulary using edit distance, which measures how many character insertions, deletions, replacements, or adjacent swaps are needed to reach a known term. For example, “toyato” is normalized to “Toyota,” “hybird” to “Hybrid,” and “sedna” to “Sedan.” I use conservative distance thresholds and require matching first letters to avoid changing unrelated normal English words.



Architecture and Technology Stack Diagram

3. Key decisions and rationale

Decision 1: AI output is constrained to a known schema.

Instead of allowing free-form AI-generated search strings to drive the system, the model

produces a typed filter object. This keeps the system safe, transparent, and testable. It also reduces prompt-injection risk and makes validation much easier.

Decision 2: Deterministic search

The actual inventory filtering happens in deterministic application code. The AI is treated as an intent extractor, not the final decision-maker. This ensures predictable results, easier debugging, and peaceful fallback behavior when AI is unavailable.

Decision 3: Stateful refinement over one-shot replacement.

The system applies each refinement to the existing search state rather than resetting the conversation every time. This mirrors how users naturally browse: "Only clean titles" followed by "Under 20k" and then "Lower mileage".

Decision 4: Transparency

The interface exposes active filters as chips and supports Undo and Reset. This is especially important in review and user trust scenarios because it shows the system what it understood.

Decision 5: Local-first resilience and safe defaults.

The prototype is usable without secrets or remote API access. This makes demos reliable and forces the architecture toward graceful degradation and a clear separation between model interpretation and trusted search logic.

4. Use simulated inventory

I generated 200 representative vehicles so the complete experience is immediately runnable. Lot numbers, bids, condition details, seller reasons, and damage severity are clearly labeled as simulated, and photos are described as representative rather than actual Copart lots.

The automated tests focus on outcomes: accumulated filters, ranges, exclusions, reset, new-search clarification, typo correction, and price ceilings. This keeps tests stable even if conversational wording changes.

5. Production considerations

To become production-facing, this project would need to evolve across several layers.

Data and inventory layer:

Replace the in-memory demo dataset with a real inventory feed. Store inventory in a scalable data store or search index such as PostgreSQL plus OpenSearch or Elasticsearch. Model freshness, versioning, and damage/title normalization as first-class concerns.

Search and ranking:

Add relevance ranking, synonyms, fuzzy matching, caching, and indexing so the experience remains fast as inventory scales. The production system needs more than simple filter matching; it needs ranking and performance optimization.

AI and safety:

Add strict validation at the API boundary, prompt-injection guardrails, structured logging, and confidence checks. The model should never be the final authority on a trusted action; it should propose a validated update.

Authentication and authorization:

If real user actions or bidding are introduced, identity and authorization become essential. Saved searches, favorites, and bid actions must be secured behind role-based access and session management.

Security and operations:

Add rate limiting, secret management, health checks, distributed tracing, monitoring, and alerting. Production systems need observability and controlled failure modes.

Business workflow extension:

Add watchlists, saved searches, notification flows, richer comparison workflows, and procurement-specific rules. This would transform the product from a demo into a real sourcing or buyer workflow.

6. Scaling as usage and inventory grow

As demand grows, the architecture should evolve in a controlled way:

- Frontend stays lightweight and focused on experience
- Search logic moves behind a dedicated backend service
- AI remains a thin interpretation layer instead of the search authority
- Inventory moves from in-memory arrays to indexed data stores
- Caching reduces repeated cost and improves latency
- Stateless backend services can scale horizontally with traffic growth

This keeps the system modular: AI, validation, search, and data storage can evolve independently over time.

Key decisions to emphasize:

- LLM for interpreting flexible language
- Deterministic code for actual filtering
- Structured, allowlisted filter contract for safety
- Multi-turn state with undo and reset
- Clarification before replacing an existing search
- Graceful fallback when NVIDIA NIM is unavailable
- Simulated data clearly disclosed as non-production
- Automated tests for important search behaviors

7. Summary:

I built a conversational vehicle search that combines natural-language understanding with a strict, deterministic filter engine. The AI interprets user intent, the system validates that intent against a structured schema, and the search layer applies it safely to the inventory. The design is resilient because the app still works with a local fallback parser, and it is ready to evolve into a production product by moving inventory and search logic behind backend services, adding identity and authorization, hardening safety controls, and scaling the data layer as volume grows.

Final takeaway:

This project demonstrates a strong architecture pattern: use AI for intent extraction, but keep business logic and enforcement in deterministic code. That balance preserves trust, explainability, performance, and safety while still delivering a natural user experience